1. Introduction. This example builds a tiny library of *lazy sequences* and uses it to print the first few even Fibonacci numbers. In barely fifty lines it exercises the Go features that have no counterpart in C, and that a literate document most enjoys showing off:

- *first-class functions and closures* — functions are values that can capture and carry local state, be passed as arguments, and be returned from other functions;
- *anonymous functions* — the **func**(...) {...} literals that make the closures above;
- *generics* — type parameters like [*A*, *B* **any**], so one *Map* works for every element type;
- *range-over-func iterators* (Go 1.23) — a function can *be* a sequence, driven directly by **for** *v* := **range** *seq*.

A sequence here is an *iter.Seq*[*V*], which is just the function type **func**(*yield* **func**(*V*) **bool**): a *producer* that hands each value to a *yield* callback and stops early if *yield* returns **false**. Because a producer is an ordinary closure, sequences can be infinite and are computed only on demand — nothing is materialised into a slice.

```
package main
import (
    "fmt"
    "iter"
)
⟨ The Fibonacci generator 2 ⟩
⟨ The Map combinator 3 ⟩
⟨ The Filter combinator 4 ⟩
⟨ The Take combinator 5 ⟩
func main() {
    ⟨ Print the first even Fibonacci numbers 6 ⟩
}
```

2. A sequence is a closure. The Fibonacci numbers are an infinite sequence, so they could never be returned as a slice; as a lazy *iter.Seq*[**int**] they are no trouble at all. *fibs* returns a closure that keeps the running pair *a*,*b* as captured local state — the kind of stateful function value C simply cannot express — and offers each *a* to *yield* until the consumer asks it to stop. The loop condition *is* the yield call: **for** *yield*(*a*) runs until *yield* returns **false**.

⟨ The Fibonacci generator 2 ⟩ ≡

```
// fibs is the infinite sequence 0, 1, 1, 2, 3, 5, 8, ... as a lazy iterator.
func fibs() iter.Seq[int] {
    return func(yield func(int) bool) {
        a, b := 0, 1
        for yield(a) {
            a, b = b, a+b
        }
    }
}
```

This code is used in section 1.

3. Composing sequences. The real payoff is composition. Each combinator takes a sequence and returns a *new* sequence — another closure — that wraps the old one. None of them does any work when called; the computation happens only when the final sequence is ranged over. This is the same pipeline style as Unix filters, but type-safe and built entirely from function values.

Map applies f to every element. Its type parameters A and B let it turn a sequence of one type into a sequence of another; the returned closure ranges over the input and yields the transformed values.

⟨ The *Map* combinator 3 ⟩ ≡

```
// Map yields f(v) for each v in s.
func Map[A, B any](s iter.Seq[A], f func(A) B) iter.Seq[B] {
    return func(yield func(B) bool) {
        for v := range s {
            if !yield(f(v)) {
                return
            }
        }
    }
}
```

This code is used in section 1.

4. *Filter* keeps only the elements satisfying a predicate. The *keep* function is supplied by the caller as an anonymous function, so the same *Filter* serves every purpose.

⟨ The *Filter* combinator 4 ⟩ ≡

```
// Filter yields the elements of s for which keep returns true.
func Filter[V any](s iter.Seq[V], keep func(V) bool) iter.Seq[V] {
    return func(yield func(V) bool) {
        for v := range s {
            if keep(v) && !yield(v) {
                return
            }
        }
    }
}
```

This code is used in section 1.

5. *Take* makes an infinite sequence finite by yielding at most n elements and then returning, which breaks the producer's loop. Without it, ranging over *fibs* would never end.

⟨ The *Take* combinator 5 ⟩ ≡

```
// Take yields at most the first n elements of s.
func Take[V any](s iter.Seq[V], n int) iter.Seq[V] {
    return func(yield func(V) bool) {
        i := 0
        for v := range s {
            if i >= n || !yield(v) {
                return
            }
            i++
        }
    }
}
```

This code is used in section 1.

6. Running the pipeline. Now we assemble a pipeline and consume it. *Filter* narrows the infinite Fibonacci stream to its even members using an anonymous predicate, *Take* caps it at eight, and the **for** *n* := **range** *Take*(*even*, 8) loop pulls the values — each computed only as it is demanded. The output is the eight numbers 0, 2, 8, 34, 144, 610, 2584, 10946.

⟨ Print the first even Fibonacci numbers 6 ⟩ ≡

```
even := Filter(fibs(), func(n int) bool { return n%2 == 0 })
for n := range Take(even, 8) {
    fmt.Println(n)
}
```

This code is used in section 1.

7. Index.

⟨ Print the first even Fibonacci numbers 6 ⟩ Used in section 1
⟨ The Fibonacci generator 2 ⟩ Used in section 1
⟨ The *Filter* combinator 4 ⟩ Used in section 1
⟨ The *Map* combinator 3 ⟩ Used in section 1
⟨ The *Take* combinator 5 ⟩ Used in section 1

Lazy Sequences in Go